EdgeFlow AI

Team Development & Setup Guide

Shared setup for all contributors | 8 September 2026

EdgeFlow AI | Team 31µF | VEDC 2026

1. Purpose and Expected Local Environment

This guide gives every team member the same local development baseline before module work is distributed. It is intentionally module-neutral: everyone should be able to clone the repository, validate contracts, start the simulation profile, run tests, inspect service health, and use a coding agent without changing shared architecture.

Setup success criterion: one command starts the simulation stack, all services become healthy, synthetic/SUMO telemetry reaches Graph-Control, a control command is produced, and the dashboard/API are reachable.

2. Prerequisites

Tool	Requirement / purpose
Git	Clone, branch and merge project code
Docker Engine or Docker Desktop	Run the microservice stack
Docker Compose v2	Use docker compose profiles and service orchestration
Python 3.11+ (recommended)	Local contract generation, scripts and non-container test utilities
Code editor / IDE	Any editor; VS Code is suitable but not mandatory
Codex or similar coding agent (optional)	Use only with AGENTS.md rules and bounded task specifications

- Windows: use Docker Desktop; WSL2 is recommended when available for a Linux-like development shell.
- macOS: Docker Desktop is sufficient.
- Linux: Docker Engine + Compose plugin is sufficient.
- Do not install SUMO, Mosquitto, database services or the web stack globally unless a module explicitly requires native debugging; the default path is containers.

3. First-Time Repository Setup

3.1 Clone and inspect the project

```
git clone <EDGEFLOW_REPOSITORY_URL>
cd EdgeFlow-AI

git status
ls
```

- Read README.md for project entry points.
- Read ARCHITECTURE.md before editing service boundaries.
- Read AGENTS.md before using Codex or another coding agent.
- Read contracts/README.md before producing or consuming telemetry/control messages.
- Read DEVELOPMENT.md for repository-specific commands once available.

3.2 Create local environment configuration

```
cp .env.example .env
```

Recommended baseline variables (names may be expanded as implementation matures):

```
EDGEFLOW_ENV=dev
EDGEFLOW_SCHEMA_VERSION=1.0
MQTT_HOST=mqtt
MQTT_PORT=1883
TELEMETRY_PERIOD_S=1
TELEMETRY_STALE_AFTER_S=3
EDGEFLOW_TOPOLOGY=four_intersection_v1
EDGEFLOW_CONTROLLER=fixed_time
LOG_LEVEL=INFO
```

- Never commit .env files containing local secrets or machine-specific overrides.
- Inside containers, use Docker service DNS names such as mqtt and api, not localhost or private LAN IPs.

4. Validate the Toolchain Before Coding

```
git --version
docker --version
docker compose version
python --version
```

Docker must be running before the next step. If docker info fails, fix Docker before debugging EdgeFlow services.

```
docker info
```

5. Start the Shared Development Stack

5.1 Simulation profile

```
docker compose --profile simulation build
docker compose --profile simulation up
```

- The first build may take longer because images and dependencies are downloaded.
- Keep the foreground session open while debugging, or add -d for detached mode.
- Expected core services: mqtt, api, graph-control, simulator, actuator and dashboard.

5.2 Inspect service state

```
docker compose ps
docker compose logs --tail=100 graph-control
docker compose logs --tail=100 simulator
docker compose logs --tail=100 api
```

All mandatory services should eventually report healthy/ready. A service crash should be debugged independently; avoid modifying unrelated services to hide the failure.

5.3 Stop and reset

```
docker compose down

# Use only when a clean local state is intentionally required:
docker compose down -v
```

The -v form removes named volumes and therefore deletes local persisted development data. Do not use it casually.

6. Contract and Interface Rules Everyone Must Follow

1. Treat /contracts as the source of truth for telemetry, control, acknowledgements, metrics, health and experiment schemas.
2. Use generated Python/TypeScript types; do not hand-copy equivalent data models into a service.
3. Use canonical IDs from the topology file, such as J1, J1_W_IN and J1_PHASE_NS.
4. Use SI units internally: metres, m/s, vehicles/s, seconds and milliseconds.
5. Preserve both timestamp_ms and simulation_time_s when applicable.
6. Publish/subscribe only to the canonical edgeflow/v1/... topic tree.
7. Do not import implementation code from another service; communicate through contracts.
8. Do not expose GPIO or SUMO-specific details to Graph-Control; use the actuator contract.
9. Do not make Graph-Control aware of whether telemetry came from SUMO, video or hardware.

7. Standard Developer Commands

The repository should expose stable Makefile targets so humans and coding agents use the same interface. Until all targets exist, use the Docker commands shown beside them.

Target command	Purpose / fallback
make contracts	Generate/validate shared types from schemas
make test	Run unit + contract tests
make integration	Run service integration tests
make smoke	Start minimal stack, inject telemetry and verify a command
make up-sim	Equivalent to docker compose --profile simulation up --build
make down	Equivalent to docker compose down
make experiment EXP=<name>	Run one experiment manifest and collect artifacts

8. Module-Neutral Development Workflow

1. Pull the latest main branch and confirm the stack is healthy before starting work.
2. Create a feature branch scoped to one module or contract change.
3. Read the relevant contract and golden fixtures before writing code.
4. Develop and run module-level tests locally.
5. Run contract tests for any producer/consumer change.
6. Run Docker integration/smoke tests before opening a pull request.
7. Document any intentional architecture change with an ADR rather than silently changing service boundaries.
8. Keep commits focused so another member or coding agent can review the change without unrelated noise.

```
git switch main
git pull
git switch -c feat/<module>-<short-description>
```

```
# work / test

git status
git add <scoped-files>
git commit -m "feat(<module>): <concise change>"
```

9. Using Codex or Similar Coding Agents

Coding agents should be treated as bounded implementation assistants, not as architecture owners. Give them explicit contracts, allowed paths and acceptance tests.

9.1 Before prompting an agent

- Ensure AGENTS.md exists and reflects the latest architecture rules.
- Point the task to the relevant schema, topology file and fixture.
- Specify the module/service and allowed files.
- Provide required behavior and acceptance tests.
- Tell the agent not to change unrelated service contracts or dependencies.

9.2 Recommended task template

```
Task:
Implement <specific behavior> in <module>.

Inputs / topics:
<canonical topic or API>

Contract:
contracts/schemas/<schema>.json

Required behavior:
- ...
- ...

Acceptance tests:
- ...
- ...

Allowed files:
services/<module>/**
tests/<appropriate-scope>/**

Do not:
- modify unrelated services
- redefine shared schemas
- change canonical IDs/units without an approved contract update
```

9.3 After an agent completes work

1. Inspect git diff; do not accept changes purely because tests pass.
2. Check for duplicated schemas, cross-service imports and hard-coded hostnames.
3. Run module tests plus contract/integration tests yourself.
4. Re-run the simulation smoke test when the change affects messaging or control flow.
5. Commit only the intended scoped change.

10. Test Layers and Golden Fixtures

```
unit
  -> contract
  -> component
  -> integration
  -> end-to-end
```

- Every shared message should have producer and consumer contract tests.
- Use shared golden fixtures instead of inventing ad-hoc test payloads.
- At minimum keep fixtures for normal telemetry, congestion telemetry, invalid telemetry, a safe phase-change command, graph state, and a directional-surge experiment.

```
tests/fixtures/
|-- telemetry_j1_normal.json
|-- telemetry_j1_congested.json
|-- telemetry_invalid.json
|-- command_j1_ns_to_ew.json
|-- graph_state_normal.json
`-- directional_surge.yaml
```

11. Common Setup and Integration Problems

Symptom	Likely cause / first check
Service cannot reach mqtt	Inside containers use MQTT_HOST=mqtt, not localhost
Graph-Control rejects telemetry	Validate schema version, canonical node/approach IDs and units
Dashboard has no live data	Check API/WebSocket health and whether state/metrics are being published
SUMO runs but signals do not change	Check control topic, command ACKs and simulator actuator mapping
Vision publishes but Graph-Control ignores it	Confirm vision uses the exact telemetry schema and canonical topology IDs
State unexpectedly becomes invalid	Check timestamp, simulation time and TELEMETRY_STALE_AFTER_S
Agent changed many unrelated files	Revert, narrow allowed paths and restate AGENTS.md constraints
Docker state is inconsistent after schema/config changes	Rebuild the affected service; use volume reset only if persistent data truly must be discarded

12. Initial Team Setup Checklist

☐ Git, Docker and Docker Compose commands run successfully.

☐ Repository cloned and README/ARCHITECTURE/AGENTS/contracts documentation read.

☐ .env created from .env.example.

☐ Simulation profile builds successfully.

☐ docker compose ps shows mandatory services running/healthy.

☐ Contracts validate and generated types are up to date.

☐ A telemetry fixture can be published/ingested.

☐ Graph-Control emits a valid ControlCommand.

☐ API health/readiness endpoints respond.

☐ Dashboard opens and receives live state when implemented.

☐ Unit/contract/integration test commands have been run at least once.

☐ Coding-agent task template and review workflow understood.

13. What Not to Block On During the Hardware-Less Phase

- Raspberry Pi-specific NCNN optimization or Pi Camera configuration.
- Final Homography calibration on a physical model.
- GPIO, relay or physical traffic-light wiring.
- ESP32, IR sensors, ESP-NOW or TFLite Micro.
- PressLight/MARL/Attn-CommNet before Max-Pressure is benchmarked.
- Perfect visual design of the dashboard before end-to-end telemetry/control works.

The first two weeks are successful when the software architecture is stable, the digital twin closes the control loop, contracts are enforced, and the same stack is ready to accept vision and later physical adapters.